

SILAB®

Änderungen/Neuerungen in SILAB® 3.0

Inhalt

1	INKOMPATIBLE ÄNDERUNGEN	3
2	KLEINERE VERBESSERUNGEN UND NEUERUNGEN.....	4
2.1	Konfigurationsdateien	4
2.2	Benutzeroberfläche von SILAB, SILABExplorer	5
2.3	DPU's	5
3	DATENBASIS SCNX	6
4	FAHRDYNAMIK VDS	9
5	SILABAEEDIT.....	9
6	VISUALISIERUNG SGE	10
6.1	SILABReplaySGE	11
7	REALISTISCHE KARTENDARSTELLUNG (DPUBIRDS EYE).....	11
8	SOUND UND MULTIMEDIA	12
9	VERKEHRSSIMULATION TRFX	13
9.1	Neue Fahrzeugmodelle	13
9.2	Erweiterte Defaultparameter für Verhaltensmodelle	14
9.3	Neue Verhaltensmodelle	15
9.3.1	EXWDM	15
9.3.2	EXSTDGGlobalS	15
9.4	Modulübergreifender Verkehr	15
10	ENTWICKLERPAKET	16

1 INKOMPATIBLE ÄNDERUNGEN

- Die Objektnamen der Ortsschilder und der Wegweiser wurden vereinheitlicht und an die Namensgebung für andere Schilder angepasst. Die Liste aller geänderten Ortsnamen finden Sie im separaten Dokument „SILAB 3.0 Neu nummerierte Verkehrszeichen“.
- Die Anpassung der Objektnamen kann durch ein Perl-Skript automatisiert werden. Dies wird in der „SILAB Version 3.0 - Installationsanleitung“ beschrieben.
- Falls Sie das Perl-Skript nicht verwenden möchten, muss
 - Bei im Skript platzierten Schildern (außerorts, Course) der Name angepasst werden
 - In SILABAEEdit platzierte Schilder (innerorts, Area2) müssen entfernt und neu eingefügt werden.
- SILAB liest keine grafischen Modelle im VEC-Format mehr ein (*.vec). Falls Sie noch solche Objekte haben, können Sie diese mit Hilfe des Programms SGEObjectPrev öffnen und als *.sig Datei abspeichern. Dadurch wird das grafische Objekt in das aktuelle SIG-Objektformat konvertiert.
- Einige veraltete grafische Objekte wurden aus der SILAB-Distribution entfernt. In der folgenden Tabelle werden die entfernten Objekte und eine Empfehlung für den Ersatz aufgeführt.

Alte Objekte	Ersatz-Objekte
Headlights.*	keine, wird nur für die veraltete Visualisierung SIG benötigt
instruction.*	keine, bei Bedarf können die alten Objekte ins Projects-Verzeichnis kopiert werden
missing.*	keine, wird nur für die veraltete Visualisierung System3DGraphics benötigt
cars.Golf.*	cars.GolfII.*
cars.Simple.*	cars.GolfII.*
environment.area.*	keine, wird nur für die veraltete Datenbasis SCN benötigt
environment.pampa.*	keine, wird nur für die veraltete Datenbasis Scenery benötigt
environment.post.*	SCNX.road.superstructures.objects.*
environment.road.CT_XX.*	keine, wird nur für die veraltete Datenbasis Scenery benötigt

environment.tree.*	environment.lgfarmed.tree*, environment.lgwooded.*
obstacles.cars.*	area2.cars.*
sign.osiwei.*	sign.310~ortsname (siehe Kap. 3)
sign.trafficlight.trafficlight*	wird nur für die veraltete Datenbasis SCN benötigt. SILABAEEdit verwendet die Objekte sign.trafficlight.TLX*.

- Wegen zahlreicher Änderungen an den Grafikressourcen ist das Update der Vorversion 2.5 etwas komplizierter. Die genaue Vorgehensweise ist in der „SILAB Version 3.0 – Installationsanleitung“ beschrieben.
- In der *DPUSGEVidGen* wurde der Parameter *ViewID* zur Auswahl des aufzuzeichnenden *SGEView* ersetzt durch den neuen Parameter *InternalSource*. Für *InternalSource* muss der Instanzname der *DPUSGEView* oder *DPUBirdsEye* angegeben werden, deren Grafikausgabe im Video aufgezeichnet werden soll. (Weitere Informationen hierzu in Kap. 6.)
- Der DPU-Parameter *AntiAlias* zur Konfiguration der Kantenglättung kann nun nicht mehr für jede *DPUSGEView*-Instanz separat angegeben werden, sondern er wird nun für alle Ansichtsfenster auf einem Computer global in der *DPUSGEWorld* festgelegt.
Bitte ersetzen Sie in den Konfigurationsdateien alle Angaben zu „SGEView.AntiAlias“ oder „SIGView.AntiAlias“ durch „SGEWorld.AntiAlias“. Dies gilt insbesondere für die Datei
(\SILAB\)\lib\configurations\system\SYSSGEPARAMS.inc.

2 KLEINERE VERBESSERUNGEN UND NEUERUNGEN

2.1 Konfigurationsdateien

- In SILAB-Konfigurationsdateien (*.cfg, *.inc) und in grafischen Objekten (*.sge, *.sig) können nun auch **mehrzeilige Kommentare** verwendet werden. Hierzu wird die aus der Programmiersprache C bekannte Syntax `/* ... */` verwendet:

```
DPUTest Test {
    A = 4;
    /* X = 1;
       Y = 2;
       Z = 3; */
};
```

Die bisher bekannten, einzeiligen Kommentare können unverändert weiterhin benutzt werden.

- Für **DPUs** können nun **Aliasnamen** verwendet werden:

```
DPUAmplifier Offset
{
    Computer = {RECHNER1};
    Alias = {MeineLieblingsDPU};
    Index = 15;
```

```

    Offset = 1.0;
};

Connections =
{
    Sinus2.Out <-> Offset.In,
    MeineLieblingsDPU.Out <-> Mixer.In
};

MeineLieblingsDPU.Multiplikator = 3;

```

- Die DPU-Aliasnamen können bei Variablenverknüpfungen und bei einer Variableninitialisierung im DPU-Pool genau wie der ursprüngliche DPU-Name verwendet werden.
- Um die Konfiguration möglichst übersichtlich zu halten, empfehlen wir, DPU-Aliasnamen nur sparsam zu verwenden. In der SILAB-Standardkonfiguration werden sie lediglich dazu eingesetzt, um die Kompatibilität von Strecken über verschiedene SILAB-Versionen hinweg zu erhöhen.
- Dazu werden für einige DPUs der Standardkonfiguration (z.B. *DPUSGEView*) zusätzlich zu ihrem eigentlichen DPU-Instanznamen (*SGEView*) noch ein DPU-Aliasname definiert, die gleich lautet wie der DPU-Instanzname in früheren SILAB-Versionen (in diesem Fall *SIGView*).

2.2 Benutzeroberfläche von SILAB, SILABExplorer

- In SILAB und in SILABExplorer werden nun zusätzliche Informationen zu DPUs und Variablen angezeigt, wenn Sie mit der Maus im Explorer über diesen verweilen.



- Für DPUs, die in einem Thread laufen, wird zusätzlich der Name dieses Threads angezeigt (im Bild oben: **amploc** läuft auf Rechner **LOCALHOST** im Haupt-Thread. Die DPU **amprem** läuft auf Rechner **LOCALHOST** im Thread **THREAD**.)
- SILABExplorer kann nun SILAB auf dem Zielrechner starten, falls es noch nicht läuft. Dazu muss allerdings der SILABButler auf dem Zielrechner gestartet sein.

2.3 DPUs

- DPUFilterBiQuad: Hochpass/Tiefpass-Filter mit einstellbarer Grenzfrequenz

3 DATENBASIS SCNX

Die folgenden Neuerungen können nur mit der Visualisierung SGE verwendet werden:

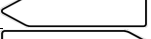
- **Bodennebelfelder** können eingefügt werden. Hierzu kann der Landschaftsgenerator „LandscapeTypeExtra GroundFog“ verwendet werden.
- **Laubwald** kann erstellt werden. Hierzu kann der Landschaftsgenerator „LandscapeType(Left/Right) LeafForest“ verwendet werden.
- Für viele **Landschaftsgeneratoren** sind nun **zusätzliche Parameter**, wie z.B. Objdist, verfügbar, womit diese noch detaillierter konfiguriert werden können.
- Die **Größe von Objekten**, die durch einen Landschaftsgenerator platziert werden, kann nun **variiert** werden. Dazu können die neuen Landschaftsgenerator-Parameter „MinScale“ und „MaxScale“ bzw. „MinScaleZ“ und „MaxScaleZ“ verwendet werden.
- Die **Größe von Bäumen**, die mit den Landschaftsgeneratoren Hedge, Alley1, Alley2, LeafForest und Forest platziert werden, sowie die Höhe von Weingärten (Vineyard) werden nun leicht **variiert** (um +/- 25%), um einen lebendigeren Eindruck zu erzielen.
- Längere **Straßentunnel** auf Überland- oder Autobahnabschnitten können mit den grafischen Objekten environment.road.tunnel_entry_* und den neuen SCNX-Objektbändern TunnelInside_2, TunnelTop_8, TunnelLight und TunnelVentilation realisiert werden.
- Um den Übergang zwischen Straße und der umgebenden Landschaft zu verbessern, kann die **Schulter** mit dem Parameter *<Transparenz>* teilweise **transparent** dargestellt werden. Mögliche Werte sind im Bereich 0 (harter Übergang) bis 1 (sehr weicher Übergang mit starker Transparenz).
Anmerkung: Die Transparenz wird nur beim Einsatz der Visualisierung SGE und nicht im „Nachtfahrt“-Modus dargestellt (SGEWorld.RenderMode muss 1..4 sein, siehe Anleitung zur DPUSGEWorld). Andernfalls ergibt sich immer ein harter Übergang.

Die Definition erfolgt im *CourseType*-Abschnitt:

CourseType *<Name Querschnittsprofil>*

```
{
    Shoulder = {<ShoulderD1>, <ShoulderD2>, <ShoulderH>,
               <Belag>, <Transparenz>};
};
```

- In der DPUSCNX wurde der Ausgang „ModuleInstanceCounter“ eingefügt, der – von 0 beginnend – bei jedem Modulwechsel des EGO-Fahrers hochgezählt wird.
- **Automatische Schildergenerierung:** Verkehrsschilder – auch komplexe, aus mehreren Einzelschildern und Zusatztafeln zusammengesetzte – lassen sich sehr einfach über eine automatische Schildergenerierung erzeugen.
- Flexibel gestaltbare **Wegweiser** stehen zur Verfügung. In der folgenden Tabelle sind alle verfügbaren Wegweiser beschrieben.

Symbol	Objektname	Beschreibung
	418_10~ziel	Wegweiser auf Landstraßen
	418_20~ziel	
	430_10~ziel/W	Wegweiser zur Autobahn
	430_20~ziel/W	
	432_10~ziel	Wegweiser innerorts
	432_20~ziel	
	434_10~zielgerade L	
	~ziellinks L	
	434_20~zielgerade L	
	~zielrechts R	
	434_30~ziellinks L	
	~zielrechts R	
	434_31~zielgerade L	
	~ziellinks L	
	~zielrechts R	
	438_10~zielgerade R ~ziellinks R	
	438_20~zielgerade L ~zielrechts L	
	438_30~ziellinks R ~zielrechts L	
	439_10~ziellinks R ~zielgerade~zielrechts L	
	439_11~ziellinks R ~zielgerade~zielrechts L	
	439_12~ziellinks R ~zielgerade~zielrechts L	Wegweiser am Kreisverkehr
	440_30~ziellinks LW ~zielgerade ~zielrechts LW	Wegweiser zur Autobahn
	448~zielabfahrt W	Vorankündigung einer Autobahn-Abfahrt
	449~zielgerade LW ~zielabfahrt LW	Autobahn-Abfahrt

- Für die kursiv gesetzten *Ziele* können jeweils unterschiedliche Ortsnamen eingesetzt werden. Diese Ortsnamen sind als Texturen abgelegt und werden in den Verzeichnissen
(D:\SILAB\)\lib\graphics\default\texture\sign\text und
(D:\SILAB\)\Projects\Projektname\sign\text gesucht.
- Standardmäßig mitgeliefert werden die Ortsnamen Aachen, Berlin, Braunschweig, Burgen, Dresden, Ebern, Eichenbach, Frankfurt, Haja, Hamburg, Hameln, Hausen, Hochdorf, Knoblach, Koeln, Lotz, Muenchen, Ostheim, Rieden, Roth, Siegburg, Stuttgart, Veitshoechheim, Wolfsburg, und Wuerzburg.
- Jedes *Ziel* ist in 6 Varianten verfügbar:
 - schwarze Schrift, zentriert: *Ziel*
 - schwarze Schrift, linksbündig: *ZielL*
 - schwarze Schrift, rechtsbündig: *ZielR*
 - weiße Schrift, zentriert: *ZieW*
 - weiße Schrift, linksbündig: *ZieLW*
 - weiße Schrift, rechtsbündig: *ZieRW*
- In der Wegweiser-Tabelle oben wird die benötigte Variante des Ziels durch die angehängten Buchstabenkürzel (z.B. „*zie*/RW“) symbolisiert.
- Informationen zu **Verkehrsregeln** können – passend zu Verkehrsschildern – in die Datenbasisdefinition aufgenommen werden. Diese Regeln werden von geeigneten Verhaltensmodellen der Verkehrssimulation beachtet und können mit dem Entwicklerpaket für SCNX (iSCNX) an beliebigen Stellen des sichtbaren Straßennetzwerks abgefragt werden.
- Informationen zur **Befahrbarkeit benachbarter Spuren** werden in die Datenbasisdefinition aufgenommen. Bei Courses erfolgt dies automatisch basierend auf den Markierungen zwischen Spuren. Bei Area2s werden diese Informationen zum Teil automatisch, zum Teil mit einem entsprechenden Werkzeug definiert. Bestimmte Verhaltensmodelle der Verkehrssimulation nutzen diese Informationen. Sie können auch über das Entwicklerpaket für SCNX (iSCNX) an beliebigen Stellen des sichtbaren Straßennetzwerks abgefragt werden.
- Einzelne Spuren, Gruppen von Spuren und ganze Streckenabschnitte können über sog. **OBS-IDs** markiert werden. Die dabei vergebenen IDs stehen als Ausgang der DPUSCNX zur Verfügung und sind eine komfortable Möglichkeit, den Fahrer in einem bestimmten Teil eines Streckenabschnitts zu lokalisieren und Auswertungsdateien zu segmentieren. Diese Informationen können auch über das Entwicklerpaket für SCNX (iSCNX) an beliebigen Stellen des sichtbaren Straßennetzwerks abgefragt werden.

4 FAHRDYNAMIK VDS

- Standardmäßig steht nun das einfach konfigurierbare Einspur-Modell VDS zur Verfügung.
- VDS ermöglicht die Simulation von unterschiedlichen Ego-Fahrzeugen über einfach anzupassende Konfigurationsdateien. Für Mittelklasse-Wagen, Transporter und LKW sind bereits einige Konfigurationen im Lieferumfang. Diese befinden sich im Verzeichnis lib\configurations\VDS.
- Einstellungen zum Fahrzeug- und Motorverhalten lassen sich über diverse Parameter in der Textdatei vornehmen. Hierzu gehören Basisdaten des Fahrzeugs, wie zum Beispiel Größe und Gewicht, aber auch Kennfelddaten des Motors, Übersetzungsverhältnisse der Gänge und Schaltpunkte sind frei einstellbar.
- Weitere Informationen finden Sie in der Dokumentation DPUVDSST4W.

5 SILABAEDIT

- Über „Ansicht/Grid“ kann jetzt ein **Gitternetz** mit einstellbarer Maschenweite eingeblendet werden. Mit dem Menüpunkt „Ansicht/Auf Grid einrasten“ (oder über den entsprechenden Menü-Button oder über die Tastenkombination „Strg + G“) wird aktiviert/deaktiviert, dass der Cursor bei der Bewegung auf das Gitter einrastet.
- Mit dem **Mausrad** kann jetzt gezoomt werden. Dabei bleibt die Bildschirmposition des Teils der Zeichnung, der unter dem Cursor liegt, unverändert.
- Die **Darstellungsgeschwindigkeit** wurde verbessert. Zoomen und Scrollen einer komplexen Zeichnung ist jetzt deutlich schneller.
- Durch das Einfügen eines **Höhendaten**-Objekts können nun Kreuzungsbereiche modelliert werden, die sich am Hang befinden. Die Höhendaten müssen als Bild (im Graustufen-Format) vorliegen.
- Als Layout-Figuren stehen auch **Kreisbögen** zur Verfügung.
- Die Orientierung von Layout-Kreisen und Kreisbögen kann umgekehrt werden.
- Neu eingefügte Querschnitte können direkt auf Layout-Figuren verankert werden.
- Bereits eingefügt **Querschnitte** können mit einem Klick **dupliziert** werden.
- Es gibt ein neues Werkzeug, um alle **Spuren zwischen zwei Querschnitten automatisch** einzufügen.
- Informationen über die **Befahrbarkeit von benachbarten Spuren** können über ein eigenes Werkzeug definiert werden.
- Als **innere Ränder** können **beliebige Polygone** mit optional ausgerundeten Ecken verwendet werden.
- **Äußere Ränder** müssen nicht mehr direkt von Port A zu Port B gezeichnet werden, sondern sie können nun wie folgt aufgeteilt werden:

- Rand 1 führt von Port A zu Querschnitt Q
- Rand 2 führt von Querschnitt Q zu Port B.
(Eine Aufteilung in mehr als zwei Ränder ist ebenfalls möglich, und die Ränder 1 und 2 müssen nicht den gleichen Bodenbelag verwenden).
- Die Breite und ein Querversatz von Längsmarkierungen kann eingestellt werden.
- Als Längsmarkierungen stehen jetzt auch Radfahrerfurten zur Verfügung.
- **Komplexe Schilder** (mit Zusatzschildern und –tafeln) sowie **Wegweiser** werden über einen komfortablen Dialog eingefügt. Dabei werden auch die entsprechenden Informationen zu Verkehrsregeln definiert.
- Das beim Export automatisch generierte **Testscript** benutzt jetzt die SILAB Standard-Konfigurationsarchitektur.
- Falls der **Export** wegen eines Triangulierungsproblems fehlschlägt, können **zusätzliche Informationen** in die Zeichnung eingeblendet werden.
- Mit dem Tool „**SILABACheck**“ kann leicht überprüft werden, ob sich beim Verknüpfen zweier Area2-Streckenabschnitte unerwünschte Überlappungen ergeben.
- Neue **Tastaturkürzel** beschleunigen das Arbeiten mit SILABEdit:
 - Zwischen den einzelnen Layers wechseln: F2, F3, ...
 - Layer ein/ausblenden: Strg + F2, Strg + F3, ...
 - Tools innerhalb einer Layer auswählen: 1, 2, ...
 - Die Eigenschaften des markierten Objekts können nun anstatt mit einem Doppelklick auch mit der Taste „P“ (**P**roperties) bearbeitet werden.
(Dadurch wird ein unbeabsichtigtes Verschieben des ausgewählten Objekts vermieden.) Abhängig von der Art des markierten Objekts erscheint ein Dialog, in dem die Eigenschaften geändert werden können.

6 VISUALISIERUNG SGE

- Die neue **DPUSGEProjection** kann verwendet werden, um 3D-Objektkoordinaten in 2D-Bildschirmkoordinaten einer bestimmten Ansicht (SGEView) umzurechnen.
Eine mögliche Anwendung der DPU ist die Simulation von Head-up-Displays, die über einem bestimmten grafischen Objekt der Szenerie Informationen einblenden.
- Die Visualisierung SGE wurde um eine **Winter-Szenerie** (Landschaft mit Schnee) erweitert. Details zur Aktivierung der Winter-Szenerie finden Sie im neuen Dokument „SILAB Möglichkeiten zur Szenariengestaltung“.
- Alle Schild-Texturen wurden überarbeitet und ihre grafische Gestaltung entspricht nun genau den amtlichen Richtlinien (StVO und VzKat).
- Die **DPUSCNXCameraPos** unterstützt wechselnde Kamerapositionen für die Aufnahme von Streckenvideos. Dabei werden Hedgehogs der Familie Re-

playPosition ausgelesen und ausgewertet. Es werden relative Koordinaten, die sich mit dem Fahrzeug bewegen ebenso unterstützt wie absolute Koordinaten zum Beispiel in Kreuzungen.

- Die **DPUSGEVidGen** kann nun alternativ zu einer 3D-Ansicht, die von einer DPUSGEView erzeugt wird, auch die schematische Vogelperspektive, die von der DPUBirdsEye dargestellt wird, oder eine Instrumentenanzeige, die von der DPUMMI dargestellt wird, als Video aufzeichnen. Um diese Funktion zu aktivieren, wird der Parameter „InternalSource“ der DPUSGEVidGen auf den Instanznamen der DPU gesetzt, die die aufzuzeichnende grafische Ansicht berechnet.

Beispiel:

```
DPUBirdsEye bev {
    Computer = {VIDEORECHNER};
    ...
};
DPUSGEVidGen vidgen {
    Computer = {VIDEORECHNER};
    InternalSource = bev;
    ...
};
```

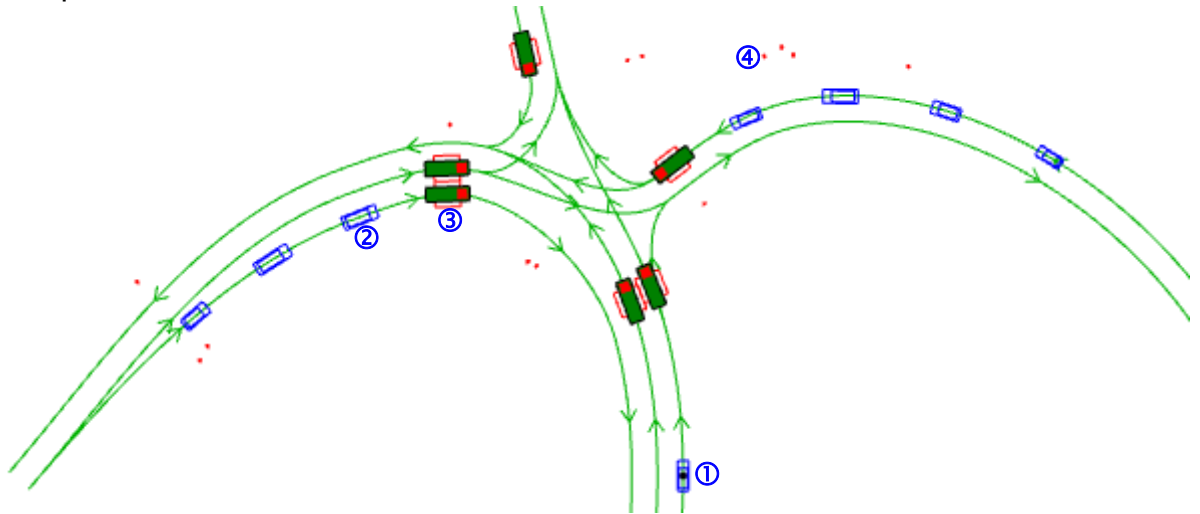
6.1 SILABReplaySGE

- Als Alternative zur Aufnahme eines digitalen Videos der Simulationsfahrt mit der DPUSGEVidGen kann nun mit der **DPUSGERecorder** ein **Replay** der Simulation **aufgezeichnet** werden. Eine Replay-Datei enthält die grafischen Modelle und Texturen der Szenerie während der kompletten Simulation und kann mit der Anwendung SILABReplaySGE abgespielt werden. Gegenüber einer Videoaufzeichnung ergibt sich der Vorteil, dass im Nachhinein noch die Kameraperspektive geändert werden kann, um beispielsweise den Ablauf einer Kollision aus der Vogelperspektive betrachten zu können.
- Neben der Fahrerperspektive können bis zu zehn **zusätzliche Kameraperspektiven** in der Replay-Datei aufgezeichnet werden, aus denen bei der Wiedergabe leicht gewählt werden kann.
- Außerdem können **Marker** in die Replay-Aufzeichnung eingefügt werden. Die Marker werden in SILABReplaySGE in einer Liste präsentiert. Der Anwender kann per Doppelklick Marker anspringen, um direkt zu einem interessanten Ausschnitt der aufgezeichneten Simulation zu gelangen.
- Die DPUSGERecorder und die Applikation SILABReplaySGE sind im neuen Zusatzpaket „Replay“ zu SILAB enthalten, das unter silab@wivw.de bestellt werden kann.

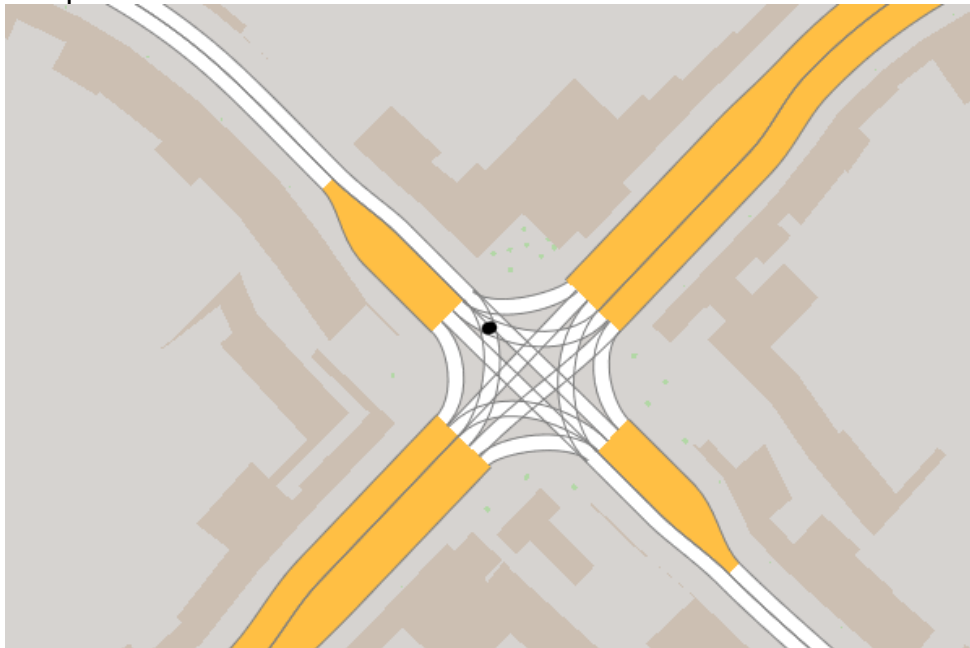
7 REALISTISCHE KARTENDARSTELLUNG (DPUBIRDSEYE)

- Die DPUBirdsEye kann wie bisher genutzt werden, um eine schematische 2D-Ansicht des sichtbaren Straßennetzwerks zu erhalten.

- Beispiel:



- In SILAB Version 3.0 wurde zusätzlich die Möglichkeit eingeführt, die DPUBirdsEye so zu konfigurieren, dass sie eine realistische 2D-Kartenansicht des sichtbaren Straßennetzes ausgibt.
- Die dargestellte Ansicht ähnelt klassischen Karten mit hoher Detailstufe, wie z.B. Stadtpläne, Internet-Kartendienste (z.B. Google Maps) oder Navigationssysteme.
- Beispiel:



- Weitere Informationen zur entsprechenden Konfiguration der DPUBirdsEye erhalten Sie in der zugehörigen DPU-Dokumentation.

8 SOUND UND MULTIMEDIA

- SILAB 3.0 enthält nun die **neue Soundsimulation SNDX**.

- Es wird eine **3D-Simulation** von Geräuschen des **eigenen Fahrzeugs** und von **Fremdfahrzeugen** durchgeführt.
 - SNDX verwendet die Sound-Schnittstelle **OpenAL** und funktioniert deshalb mit den meisten Standard-Onboard-Soundkarten, wie sie in jedem PC oder Notebook verbaut sind. Wir empfehlen allerdings, für die optimale Performance eine Creative X-Fi (oder kompatible) Soundkarte zu verwenden.
 - Im Abschnitt „Computerconfiguration“ der Versuchs-Konfigurationsdatei (normalerweise ist dieser in der Datei `SILAB\lib\configurations\COMPBase.inc` definiert) muss der neue Computer-Aliasname „**SNDX**“ vergeben werden.
 - Die DPUs **DPUSNDXListener**, **DPUSNDXEgo** und **DPUTRFSNDX** werden zur Simulation von Soundquellen benutzt und sind in der `DPUBase_SGE.inc` standardmäßig eingebunden.
 - Zusätzliche Sounddateien (im WAV-Format) können mit der **DPU-WavPlayerSNDX** eingebunden und in der Simulations-Welt 3D-positioniert werden.
 - Wenn das „SNDX“-Soundmodell verwendet wird, ersetzt die DPU-WavPlayerSNDX die alte DPUWavPlayer und die Möglichkeit, „User-Sounds“ in der DPUSoundSim einzubinden.
-
- SILAB 3.0 kann auf Wunsch zusätzlich um das neue **Sprachsynthese-Modul TTS** erweitert werden.
 - Die Sprachsynthese basiert auf dem Produkt SVox (siehe www.svox.com).
 - Es können beliebige Texte ausgegeben werden. Die Sprachqualität ist sehr gut. Standardmäßig wird die Ausgabe deutscher Texte unterstützt. Sie kann aber leicht um weitere Sprachen erweitert werden.
 - Die Sprachsynthese wird von den DPUs **DPUTTSServer** und **DPUTTSCli-entSNDX** gesteuert.
 - Texte können über einen DPU-Eingang der **DPUTTSClient** oder über Streckenevents (Hedgehogs) mit der **DPUTTSSCNX** vorgegeben werden.
 - Bei Interesse am Paket TTS wenden Sie sich bitte an das WIVW (silab@wivw.de).
 - Die neue **DPUMediaPlayer** kann verwendet werden, um während der Simulation beliebige Audio- oder Videodateien abzuspielen. Es werden alle Dateiformate unterstützt, die auch vom Windows Media Player abgespielt werden können.

9 VERKEHRSSIMULATION TRFX

9.1 Neue Fahrzeugmodelle

- Die Verkehrssimulation wurde um neue Fahrzeugmodelle erweitert. Dies sind:

- **cars.BMW320d**
 - **cars.MAN_TGA** *(Schwerer LKW, teilweise mit Anhänger)*
 - **cars.MAN_TGX** *(Schwerer LKW, teilweise mit Anhänger)*
 - **cars.MountainBike** *(Fahrrad mit animiertem Fahrer)*
 - **cars.OldBike** *(Motorrad mit animiertem Fahrer)*
 - **cars.Stadtbus** *(Öffentlicher Nahverkehr)*
 - **cars.Traktor** *(Landwirtschaftsfahrzeug)*
- Um die neuen Fahrzeugmodelle zu verwenden, muss in der Versuchskonfiguration im globalen `SILAB TRF` – Abschnitt zusätzlich die Anweisung `include SILAB_TRFX_EXT.cfg` eingefügt werden.
 - Für das Einfügen neuer Fahrzeugmodelle mit dem **SILABAE**dit in eine Kreuzung (**Area2-Knoten**) muss zusätzlich die Option *Datei → Importieren → Verkehrsdefinition...* → `SILAB\lib\aedite\trafficinfoext.cfg` ausgewählt werden.
 - Die Fahrzeugmodelle sind jeweils in **mehreren Farbvarianten** verfügbar. Eine Referenz der Eigenschaften und Screenshots dieser Modelle finden Sie in der **„SILAB Referenz TRFX“**.
 - Zusätzlich sind in der `SILAB_TRFX_EXT.cfg` neue Farbvarianten einiger bereits in SILAB 2.5 vorhandener Fahrzeugmodelle enthalten.

9.2 Erweiterte Defaultparameter für Verhaltensmodelle

Alternativ zu den Angaben der Verhaltensmodellparameter in Tupelform innerhalb des Definitionsabschnitts eines Vehicle können die Parameter auch als **Parameterzuweisung** innerhalb der **Vehicle**-Definition angegeben werden. Diese Alternative steht ebenfalls bei **VehicleX** in Kombination mit den Verhaltensschemata zur Verfügung.

Beispiel:

```
VehicleX V7R
{
    Vehicle = PKW;
    Position = (c1_102a,570,4);
    BehaviourScheme = _Standard_;
    StartSpeed = 0;
    TargetSpeed = 0;
    Flowpoints =
    {
        (ModuleTime, 0.0, Activate)
    };
};
```

In obigem Beispiel wird der Verkehrsteilnehmer V7R stehen bleiben obwohl im Verhaltensschema `_Standard_` eine Geschwindigkeit von 50 km/h angegeben wurde.

9.3 Neue Verhaltensmodelle

9.3.1 EXWDM

Das Fahrermodell ‚EXWDM‘ basiert auf einer Kombination aus einem **Fahrzeugfolgemodell** von Wiedemann¹ und dem **Spurwechselmodell MOBIL** (Minimizing Overall Braking decelerations Induced by Lane changes)². Das Spurwechselmodell kann wahlweise aktiviert oder deaktiviert werden.

Das Fahrzeugfolgemodell nach Wiedemann simuliert detailliert das **freie Fahren** und **Folgefahren** eines Fahrers, der nach Vorgabe einer Wunschgeschwindigkeit versucht diese einzustellen und zu halten. Dabei wird ein **Pendeln** um die **Zielgeschwindigkeit** ebenso simuliert wie das **Halten** eines **Abstandes** zum Vordermann beim Folgefahren.

Das Spurwechselmodell **MOBIL** wählt bei mehreren möglichen Spuren (mit gleicher Fahrtrichtung) unter Berücksichtigung des **Rechtfahrgebots** zu jeder Zeit diejenige aus die den Wünschen des Fahrers am besten entspricht und führt einen erforderlichen **Spurwechsel selbstständig** durch.

Eine ausführliche Beschreibung, sowie alle Parameter des Modells findet sich in der Dokumentation ‚Silab Referenz TRFX‘ unter Abschnitt 3.3.10.

9.3.2 EXSTDGlobalS

Das Verhaltensmodell EXSTDGlobalS ist für den modulübergreifenden Ausschmückungsverkehr konzipiert. Es beachtet grundlegende Verkehrszeichen **selbstständig**, sofern diese als **Verkehrsregeln** in das Szenario eingefügt wurden.

Folgende Verkehrsregeln/-zeichen werden ausgewertet und gemäß Ihrer Bedeutung umgesetzt:

- **Rechts vor Links,**
- **Vorfahrtsstraße,**
- **Vorfahrt achten,**
- **Stoppschilder,**
- **Ampeln,**
- **sowie Geschwindigkeitsbegrenzungen.**

9.4 Modulübergreifender Verkehr

- Zur Ausschmückung der Szenarienübergänge ist es möglich Verkehrsteilnehmer modulübergreifend zu definieren und von einem Modul in das nächste mitzunehmen.
- Hierzu steht die neue Verkehrsteilnehmerklasse **GlobalConstVehicle** zur Verfügung, welches in einem Modul definiert wird und über Modulgrenzen hinwegfahren kann. Sollen mit diesem Fahrzeugtyp inhaltliche Abläufe erstellt werden, so erfolgt dies mittels sogenannter ‚**Rollen**‘. Unter einer ‚Rolle‘ wird ein Verhaltensablauf verstanden, der beschreibt wie sich ein Fahrzeug relativ

¹ Siehe R. Wiedemann, [Simulation des Straßenverkehrsflusses](#). Schriftenreihe des Instituts für Verkehrswesen der Universität Karlsruhe, 1974.

² Siehe Treiber, M., Helbing, D.(2002). Mikrosimulation im Straßenverkehr mit einem einfachen Modell. In D. Tavanganian, R. Grützner (Eds.), ASIM 2002 – Tagungsband, 16. Symposium Simulationstechnik, Rostock, 514ff.

zu anderen Verkehrsteilnehmer verhalten soll. Dem Fahrzeug kann in einem Nachfolgemodul eine neue Rolle zugewiesen werden und diese wird übernommen, sofern die passende Voraussetzungen (wie beispielsweise der passende Fahrzeugtyp ‚PKW‘) dafür vorliegen.

10 ENTWICKLERPAKET

- Erstellen von DPUs in Java
 - DPUs können in der Programmiersprache **Java** erstellt werden. (Sie werden dann als „JPU“, Java Programming Unit bezeichnet.)
 - **JPU**s werden mit Hilfe der neuen **DPUJava** in die Simulation eingebunden.
 - Für JPUs stehen **Schnittstellen** zum Zugriff auf die Datenbasis **SCNX** und die Objektdatenbank **ODB** zur Verfügung. Diese bieten die gleiche Funktionalität wie die Schnittstellen iSCNX und ODB für mit Visual C++ erstellte DPUs.
 - Eine vollständige Dokumentation dieser Schnittstellen im javadoc-Format wird mitgeliefert.
 - Zur **Überprüfung** der JPUs kann ein Standard-Java-Compiler oder alternativ der mit SILAB mitgelieferte **SILABjavac** verwendet werden. Als Quelltext im Projektverzeichnis vorliegende Java-Klassen werden beim Start der Simulation allerdings auch **automatisch kompiliert**, so dass dies bei kleineren Änderungen nicht separat erledigt werden muss.
 - Ein beliebiger **Texteditor** (möglichst mit Syntax-Highlighting für Java) kann zum Erstellen von JPUs verwendet werden.
Bei größeren Projekten bietet es sich allerdings an, eine **IDE** (z.B. Eclipse) zum Erstellen und **Debuggen** der JPUs zu verwenden.
- Mit den neuen Makros **VARInitAID()** und **VARInitAOD()** können in DPUs, die mit Visual C++ erstellt werden, Arrays von DPU-Eingängen oder –Ausgängen angelegt werden, d.h. mehrere durchnummerierte Variablen.
- Die **DPURubyConsole** vereinfacht das Testen und Debuggen von Ruby-Skripten, die über die DPURubyScript in die Simulation eingebunden werden.
- In der Schnittstelle **iSCNX** können nun Informationen über **Verkehrsregeln** und die **Befahrbarkeit** von benachbarten Spuren an beliebigen Stellen des sichtbaren Straßennetzwerks abgefragt werden.
- Der Abschnitt „Erstellen einer DPU mit **Matlab**/SIMULINK“ im Dokument „Erweiterung um DPUs“ wurde aktualisiert. Der Vorgang wird nun für das generische Realtime-Workshop-Target beschrieben.